CUATS Hackathon 2026

19.03.2026 – 20.03.2026

Outline for the CUATS Leetcode Hackathon

- Introduction into Data Structures and Algorithms (DSA)
- Overview of the Big-O notation
- Commonly used DSA concepts
    - Arrays, Hashmaps, Dictionaries
    - Two Pointers & Binary Search
    - Linked lists
    - Trees
    - Depth-first search / Breath-first search
    - Backtracking
    - Graphs
    - Dynamic Programming
- Hackathon!

Motivation

- **Essential for Quant/Tech interviews**: Hedge funds and trading firms often ask algorithmic coding questions – your success in these interviews can hinge on your familiarity with LeetCode-style problems

- **Sharpen core algorithmic thinking**: Tackle diverse problem types (e.g., dynamic programming, graph algorithms, data structures) that boost your ability to think under time pressure

- **Optimize code performance**: Practice thinking about and writing efficient solutions. This is clearly crucial in high-frequency trading, where microseconds matter

- **Build confidence & endurance**: A 24-hour format challenges you to maintain focus, adapt quickly, and handle pressure. All are key skills for real-world trading scenarios

Overview of the Big-O notation | Introduction

Introduce & familiarize with asymptotic notation

Show how to analyse the running-time of algorithms

Overview of the Big-O notation | Introduction

Analyzing an algorithm: predicting the resources that the algorithm requires (complexity measures):

Time (CPU time)	Space (memory)	IO (bandwidth)	Energy consumption

We (normally) focus on **time behavior**

Central question: when is one algorithm better than another?

Source: University of Oxford, Ralf Hinze, Algorithms

Overview of the Big-O notation | Introduction

- The running-time is usually described in terms of the input size
- But, there are also output-sensitive algorithms (e.g., permutations, discussed later)
- How to measure the input or output size?
- Different levels of precision:
  - **rough**: number of “items” in the input (eg length of input list);
  - **precise**: number of bits needed to represent the entire input.

- How to measure the running-time?
- Different levels of precision:
  - **rough**: count number of “important” or “dominant” operations;
  - **precise**: count number of primitive operations.

Rough running times will most likely be applicable to you

Source: University of Oxford, Ralf Hinze, Algorithms

Overview of the Big-O notation | Introduction

We can analyse the

- **best-case running-time**: the shortest running-time for any input of a given size
- **average-case running-time**: the running-time for inputs of a given size on average
- **worst-case running-time**: the longest running-time for any input of a given size

We **concentrate on worst-case analysis**

- Any algorithm can be modified to have a good best-case running-time
- It may not be apparent what constitutes an “average” input. The average-case is often as bad as the worst-case

Overview of the Big-O notation | Introduction

Generally, we are interested in the order of growth.

In particular, we are prepared to ignore multiplicative constants: $\frac{1}{2}n^2$ has the same order of growth as $1000\,n^2$.

(Machines will improve constantly over time, different users have different setups with differing computing power)

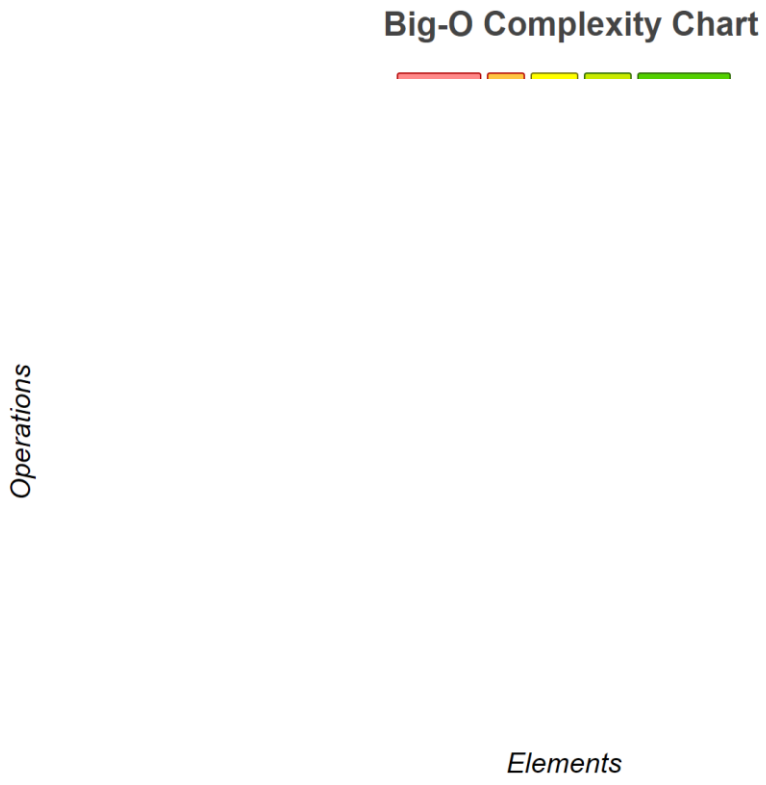
Example:
- Informally, bubble sort has a “quadratic” worst-case running-time.
- Formally, bubble sort has a worst-case running-time of $\Theta(n^2)$.

Overview of the Big-O notation | Running time comparison

A hierarchy of running-times:

- constant $\Theta(1)$: hurrah;
- logarithmic $\Theta(\log(n))$: almost constant for all practical purposes;
- linear $\Theta(n)$: good - for many problems we have to inspect the entire input;
- quadratic $\Theta(n^2)$: OK,
- polynomial $\Theta(n^k)$: hmm,
- exponential $\Theta(2^n)$: ouch.
- factorial $\Theta(n!)$: ...

Only algorithms with a polynomial running-time or better are considered reasonable.



Source: University of Oxford, Ralf Hinze, Algorithms, https://www.bigocheatsheet.com/

<BIG-O-CHEATSHEET>

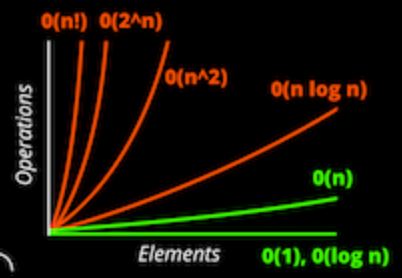
www.bigocheatsheet.com

LEGEND

TIME Complexity vs. SPACE Complexity

Good Fair Bad

Good Fair Bad



DATA STRUCTURE Operations

DATA Structure	TIME Complexity Average: Access	Search	Insertion	Deletion	TIME Complexity Worst: Access	Search	Insertion	Deletion	SPACE Complexity Worst
Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

ARRAY SORTING Algorithms

ARRAY Algorithms	TIME Complexity Best	Average	Worst	SPACE Complexity Worst
Quicksort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	Θ(n log(n))	O(n log(n))	O(1)
Heapsort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	Θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	Θ(n log(n))	O(n^2)t	O(n)
Shell Sort	Ω(n log(n))	Θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	Θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	Θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	Θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)

Overview of the Big-O notation | Final words

The Big-Oh notation hides multiplicative factors.
In algorithm design we strive for order of magnitude improvements, see quote by Christopher Strachey

Writing programs needs genius to save the last order or the last millisecond. It is great fun, but it is a young man's game. You start it with great enthusiasm when you first start programming, but after ten years you get a bit bored with it, and then you turn to automatic-programming languages and use them because they enable you to get to the heart of the problem that you want to do, instead of having to concentrate on the mechanics of getting the program going as fast as you possibly can, which is really nothing more than doing a sort of crossword puzzle.

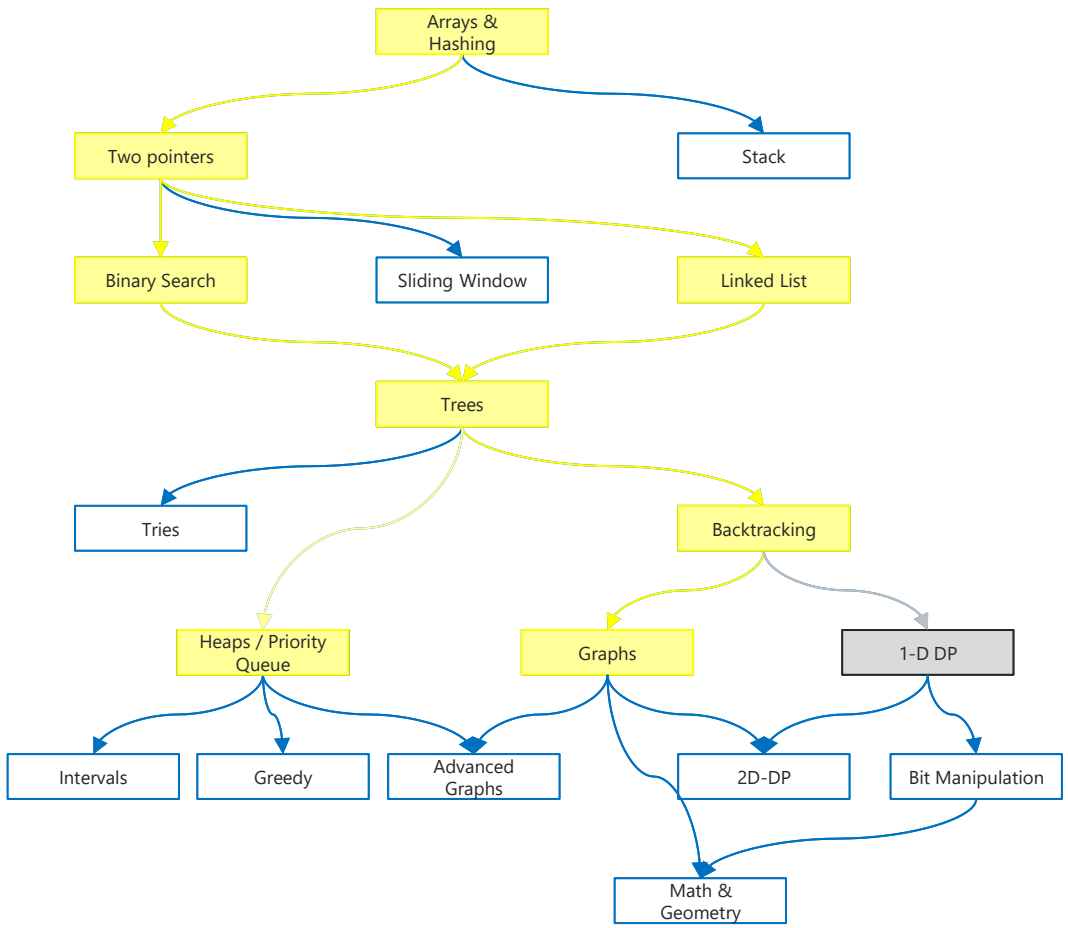
Source: University of Oxford, Ralf Hinze, Algorithms

General approach to DSA problems

Data structures and algorithms built on top of each other

Unfortunately, the Leetcode problem order DOES not account for your previous skills

Common lists to practice: Blind 75 & Neetcode 150

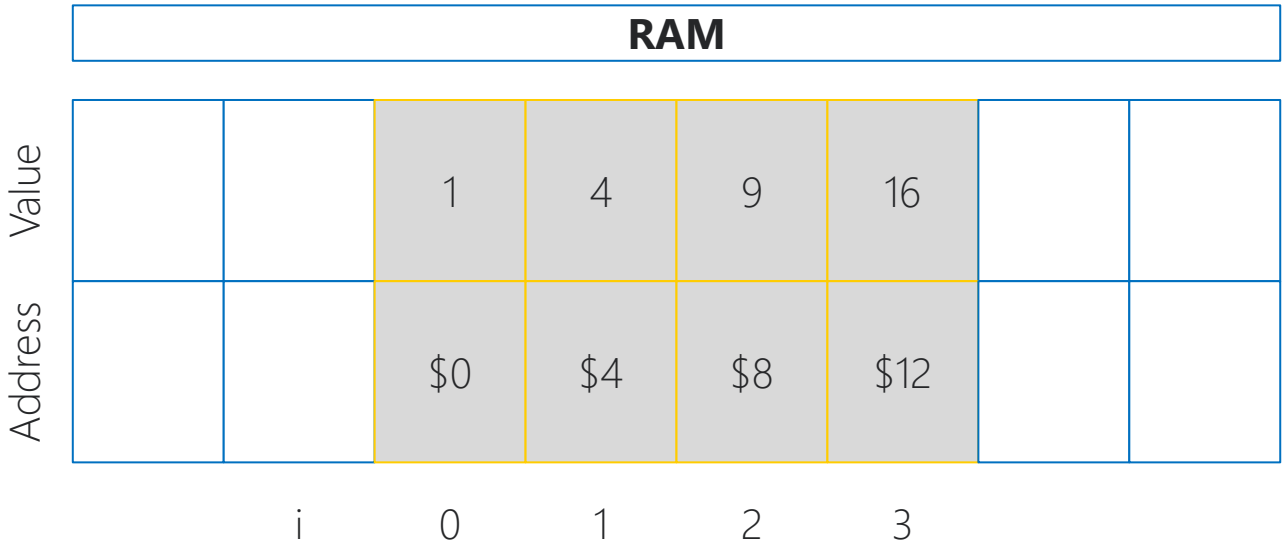


Source: Neetcode

Arrays, Hashmaps, and Dictionaries

Python uses dynamic arrays natively

Arrays are a collection of elements stored at contiguous memory locations. They allow efficient data retrieval and manipulation



Opera-tion	Big-O time	Notes
Access	O(1)	Search is O(n)
Insert	O(n)	O(1) if at the end of array (append)
Delete	O(n)	O(1) if at the end of array

Introduction into Hashsets and Hashmaps (Dictionaries)

```
names = ["alice", "bob", "charlie", "bob", "daniel"]
```

Hashmap	
Key	Value (count)
“alice”	1
“bob”	2
“charlie”	1
“daniel”	1

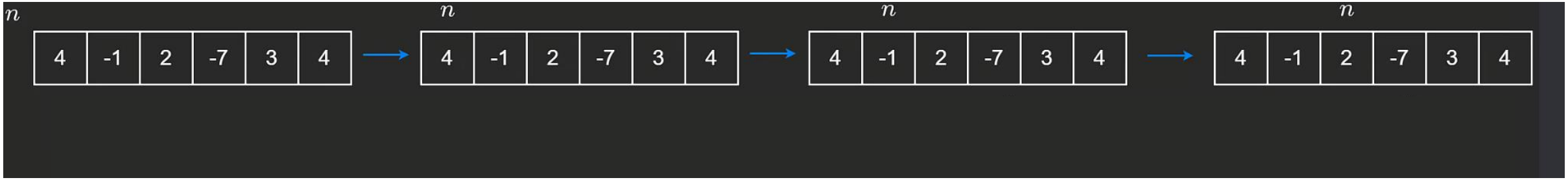
Hashset
“charlie”
“alice”
“daniel”
“bob”

```
hashmap = {} #basically a dict
hashset = set()
```

Opera-tion	Big-O time	Notes
Access	O(1)	NO order is kept. Impractical for (most) sorting tasks etc.
Insert	O(1)	
Delete	O(1)	

Kadane's Algorithm

Task: Find the **maximum consecutive sum** in an array




```python
def bruteForce(nums):
    maxSum = nums[0]
    for i in range(len(nums)):
        curSum = 0
        for j in range(i, len(nums)):
            curSum += nums[j]
            maxSum = max(maxSum, curSum)
    return maxSum
```

```python
def kadanes(nums):
    maxSum = nums[0]
    curSum = 0
    for n in nums:
        curSum = max(curSum, 0)
        curSum += n
        maxSum = max(maxSum, curSum)
    return maxSum
```

Source: Neetcode

Practice Questions 1

http://bit.ly/4cYquOI

Two pointers & Binary search

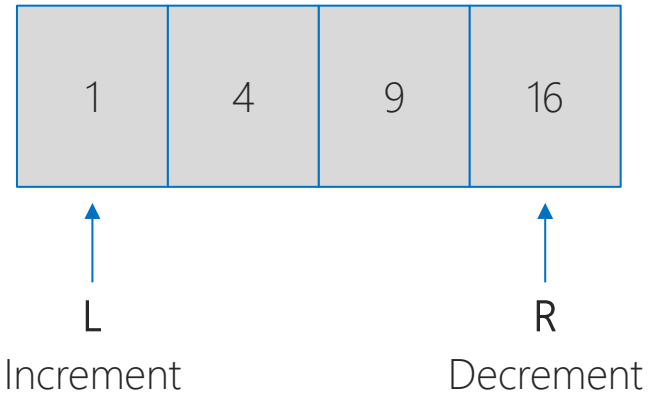
Two Pointers

Have a L (left) pointer and a R pointer, both starting at some indices of the array.

They don't always have to start at the beginning of the array. The L and R pointers can start at any index of the array.

Start the L pointer at 0 and R pointer at arr.length - 1 and increment either the L, or decrement R or both depending on the conditions given in the problem.

This repeats until the pointers meet each other.



Example: Verify palindrome

```
def isPalindrome(word):
   L, R = 0, len(word) - 1
      while L < R:
         if word[L] != word[R]:
            return False L += 1 R -= 1
   return True
```

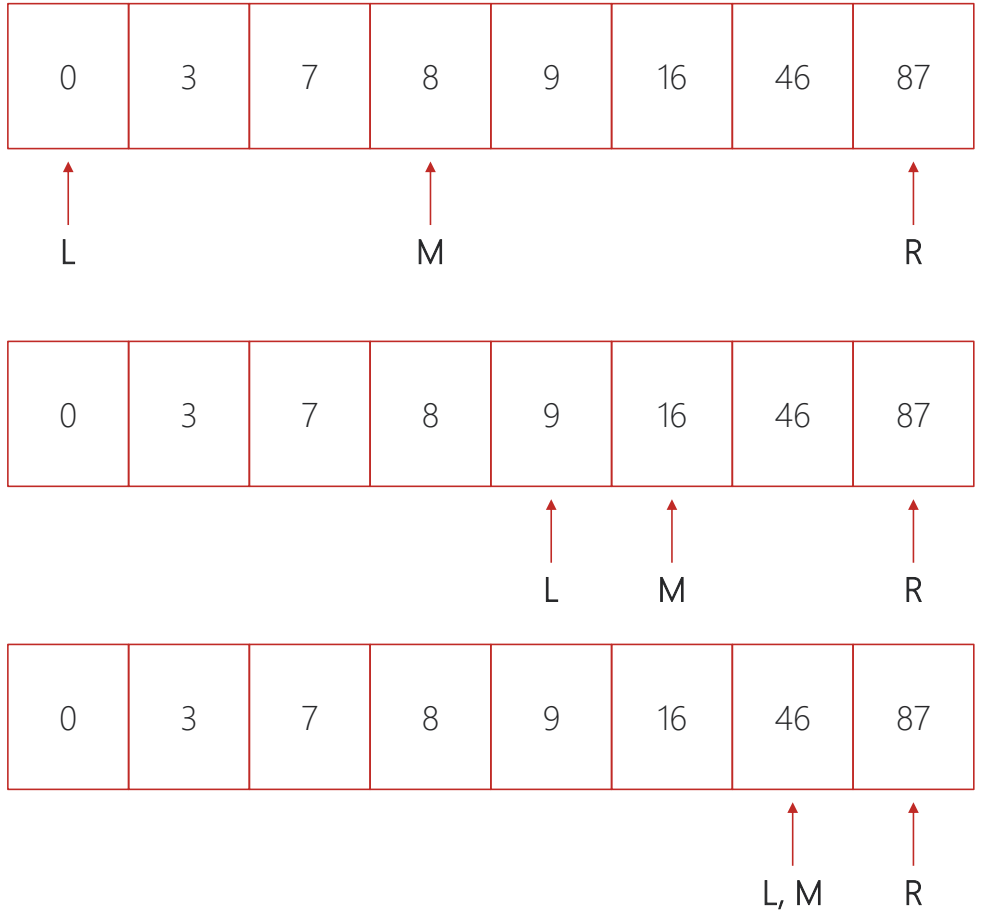
Source: Neetcode

Binary search

Binary search is an efficient way of searching for elements within a sorted array. Typically, we are given a sorted array, and a target element to search for:

```
arr = [0, 3, 7, 8, 9, 16, 46, 87]
def binarySearch(arr, target):
    L, R = 0, len(arr) - 1

    while L <= R:
        mid = (L + R) // 2
        if target > arr[mid]:
            L = mid + 1
        elif target < arr[mid]:
            R = mid - 1
        else:
            return mid
    return -1
```

Example: Target: 46



Source: Neetcode

Practice Questions 2

http://bit.ly/4cYquOI

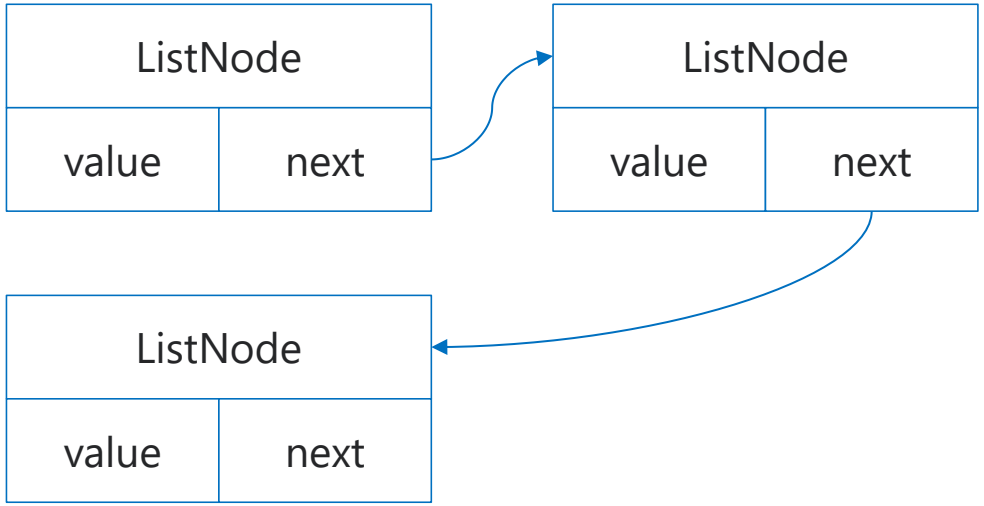
Linked lists

Linked lists

A linked list is another data structure that is like an array in the sense that it stores elements in an ordered sequence.

The main difference is that linked lists are made up of objects called ListNodes. This object contains two attributes:

- value – This stores the value of the node. It could be a character, an integer, etc.
- next – This stores the reference to the next node in the linked list.




```
class ListNode:
   def __init__(self, val):
      self.val = val
      self.next = None
```

```
ListNode1.next = ListNode2
ListNode2.next = ListNode3
ListNode3.next = null
```

Example of walking through linked list:

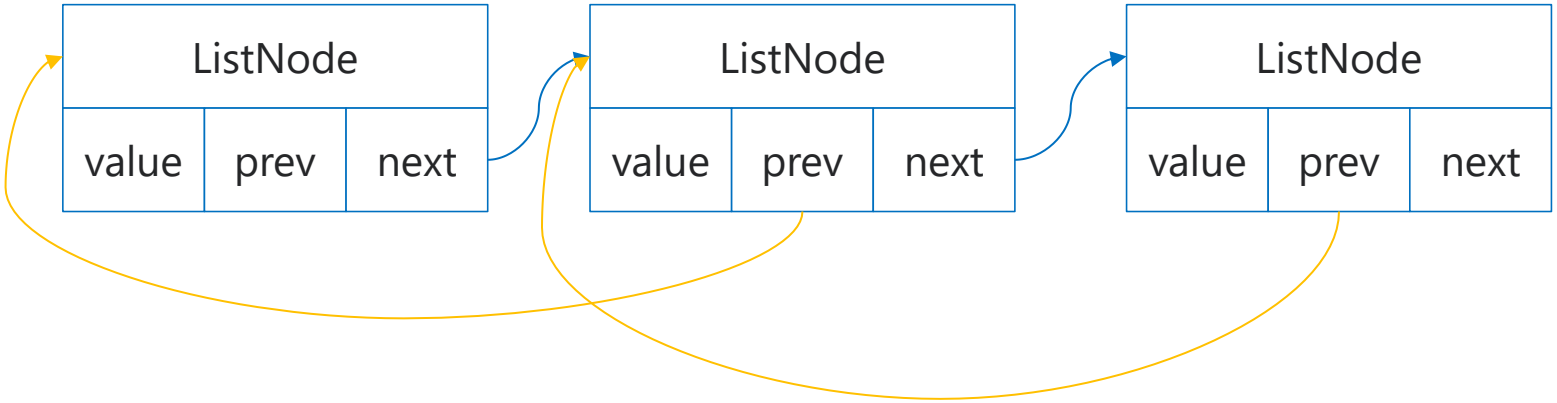
```
cur = ListNode1
while cur:
   cur = cur.next
```

Source: Neetcode

Linked lists

A linked list is another data structure that is like an array in the sense that it stores elements in an ordered sequence.

Alternatively, a linked list can also have multiple pointers. A linked list with 2 pointers is commonly used as a double-linked list



Opera-tion	Big-O time	Notes
Access	O(n)	
Search	O(n)	
Insert	O(1)*	Only if the reference to the node exists!
Delete	O(1)*	Only if the reference to the node exists!

Source: Neetcode

Practice Questions 3

http://bit.ly/4cYquOI

Trees

Trees | Binary Trees

Similar to linked lists, binary trees are another data structure that involve nodes and pointers.

With linked lists, we connected nodes in a straight line with *next* and *prev* pointers.

Nodes in a binary tree also have at most two pointers, but are called left child and the right child pointers. The first node in a binary tree is referred to as the root node.

The value of a node can be any data type. A TreeNode class would look like the following. This implementation is similar to a ListNode, except these nodes are considered **children**.

```
class TreeNode:
   def __init__(self, val):
      self.val = val
      self.left = None
      self.right = None
```

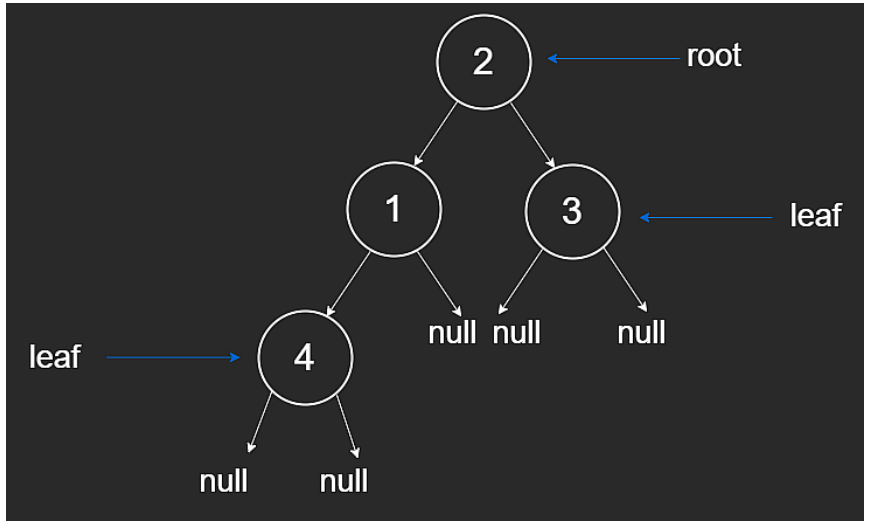
Source: Neetcode

Trees | Binary Trees

Terminology

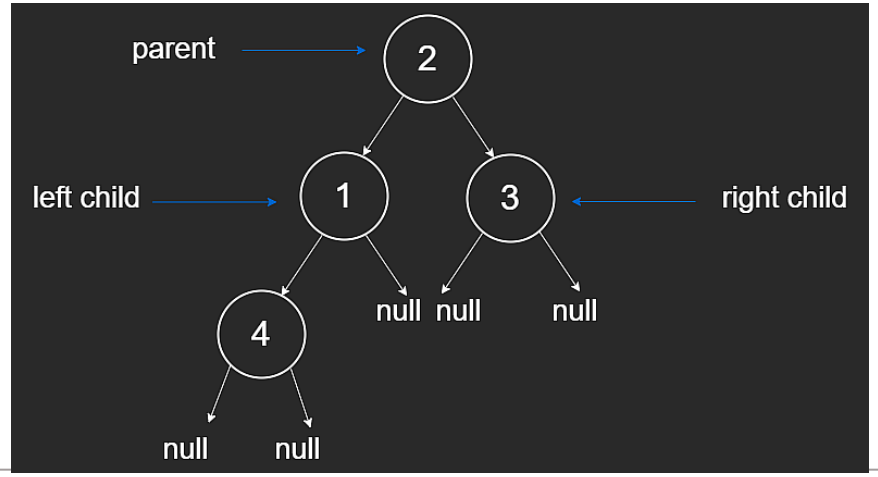
Root Node

Root node is the highest node in the tree and has no parent node. All of the nodes in the tree can be reached by the root node.



Leaf Nodes

Leaf nodes are nodes with no children. The nodes at the last level of the tree are guaranteed to be leaf nodes but they can also be found on other levels.



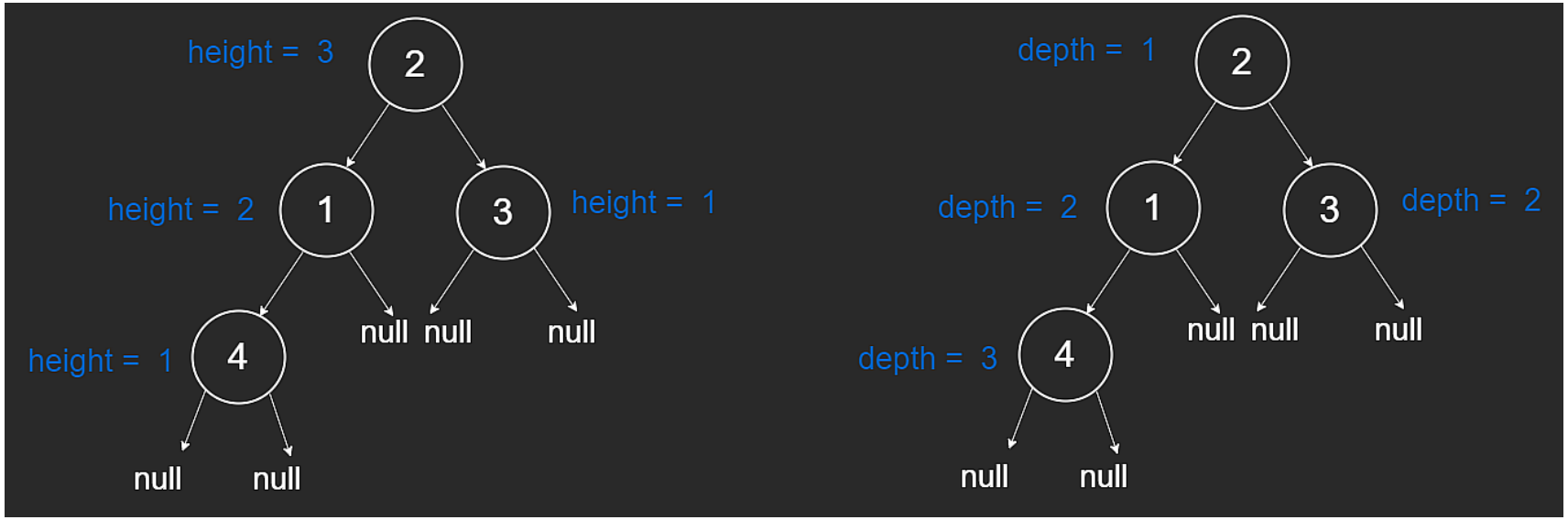
If a node does not have any children, it is classified as a **leaf node**. If a node has even a single child, either left or right, it would be classified as a **non-leaf node**.

Source: Neetcode

Trees | Binary Trees

The **height** of a binary tree is measured from the root node all to way to the lowest leaf node, just like the height of anything in real life. The height of a single node tree is just 1, if the node itself is counted, or 0 if not.

The **depth** of a binary tree node is measured from itself all the way up to the root. As observed in the visual below, the depth at the root node is 1, with it increasing as we go down. Measure depth at a given node by looking at how many nodes are above it, including the node itself.



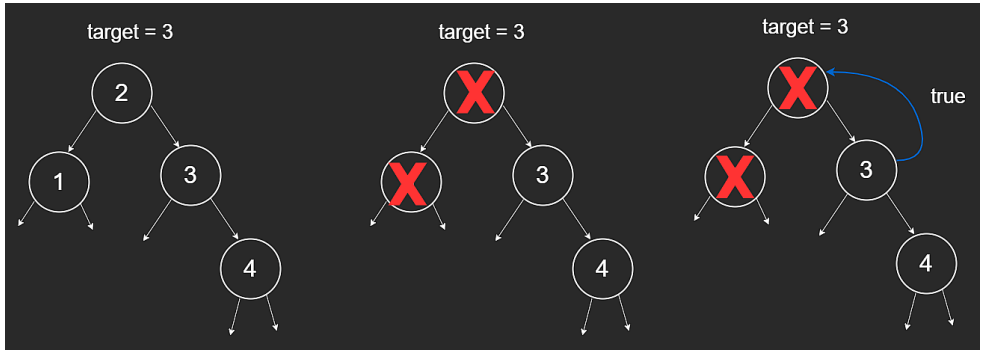
Source: Neetcode

Trees | Binary Trees | Time complexities

If we have a **balanced binary tree**, the search algorithm will run in O(log n) time. Balanced binary tree means that the height of the left-subtree is equal to the height of the right-subtree, or there is a difference of 1.

In a balanced tree, we can eliminate half the nodes each time

In the worst case we have a skewed tree, where the height of the tree is equal to the number of nodes. In this case, the time complexity will be O(n).




```
def search(root, target):
    if not root:
        return False
    if target > root.val:
        return search(root.right, target)
    elif target < root.val:
        return search(root.left, target)
    else:
        return True
```

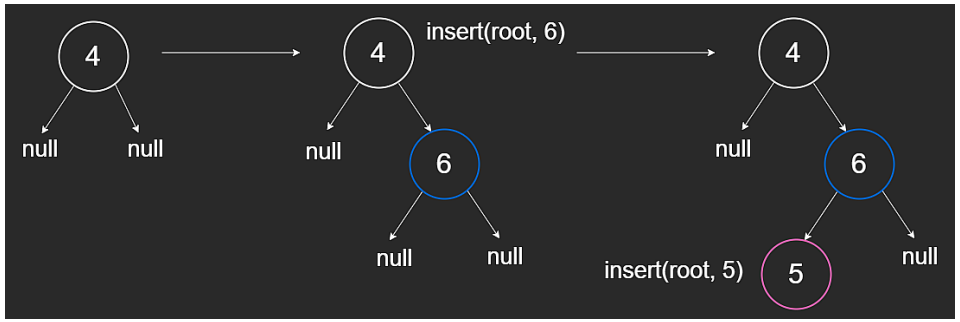
Source: Neetcode

Trees | Binary Trees | Insertion and removal

One of the main benefit of using BSTs over sorted arrays is that **removal and insertion in O(log n) time**. This assumes that the tree is balanced (otherwise, the worst-case would be O(n).

To insert a new node into the BST, we first have to traverse the BST to find the right position, and then insert this node.

We must make sure we maintain the sorted property of the BST, just like we would with a sorted array.




```python
# Insert a new node and return the root of the BST.
def insert(root, val):
    if not root:
        return TreeNode(val)

    if val > root.val:
        root.right = insert(root.right, val)

    elif val < root.val:
        root.left = insert(root.left, val)

    return root
```

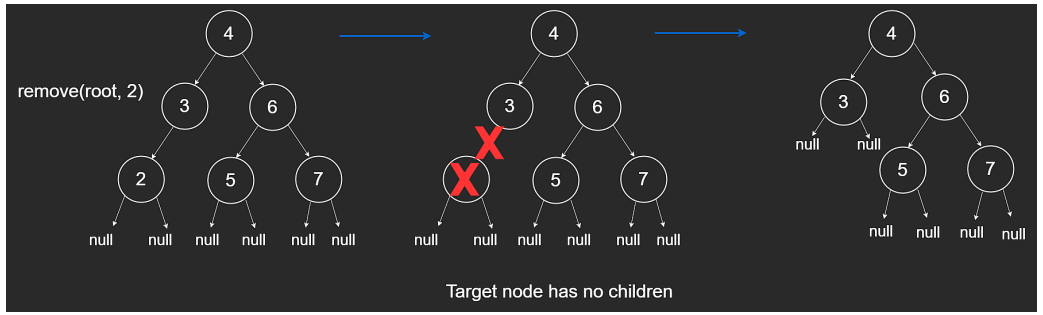
Source: Neetcode

Trees | Binary Trees | Insertion and removal

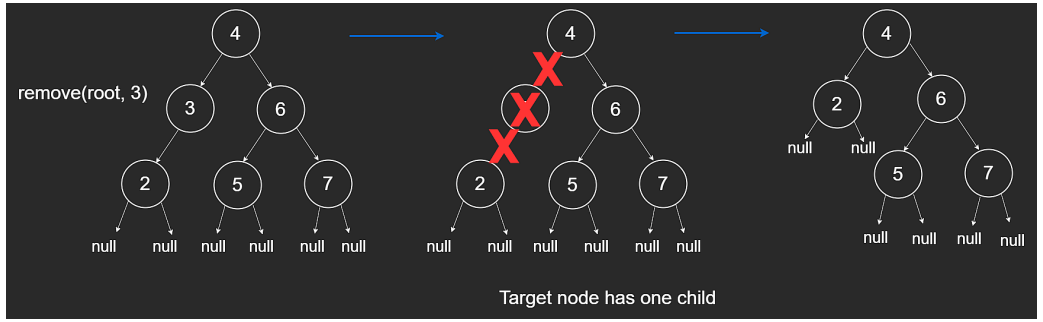
The time complexity of inserting and removing from a BST is proportional to the height of the tree. This is because we are essentially running a binary search on the tree until we reach the target position to insert into, or the target node to remove.

The time complexity of these operations is O(log n) if the tree is balanced. However, if the tree is unbalanced, the time complexity can be O(n) in the worst case.

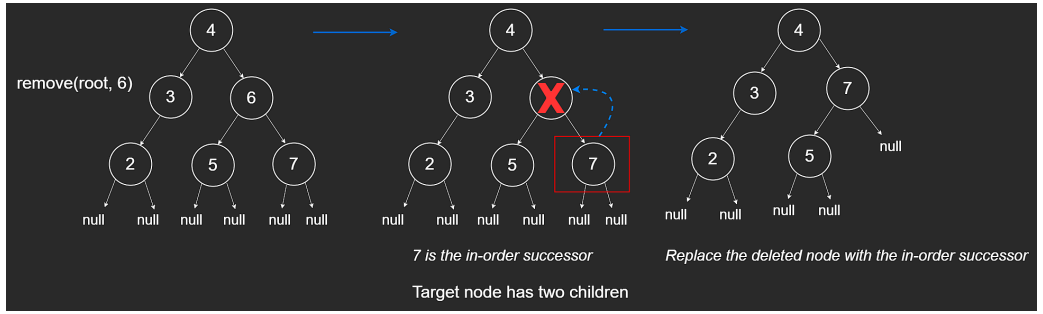
1. Case: No children



2. Case: One child



3. Case: Two children



Source: Neetcode

Trees | Depth-First Search (DFS)

Depth First Search (DFS) is one of the most common algorithms in coding interviews. It is commonly used to traverse trees and graphs.

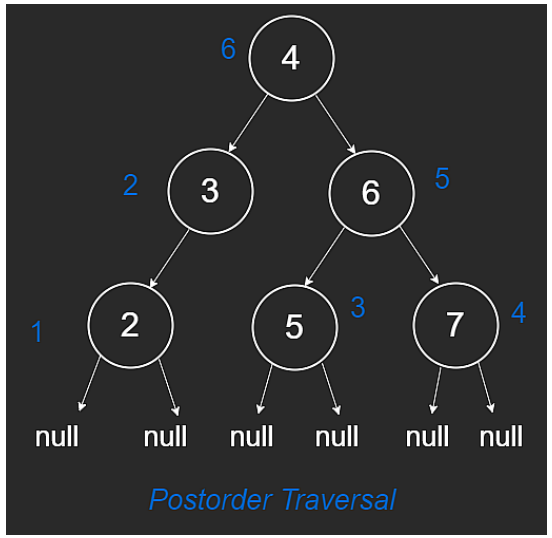
If we want to traverse an entire tree, we have to visit every node (O(n) time). One way to accomplish this is by using depth-first search.

The idea is we pick a direction and keep following pointers as far down left as we can go until we reach null. Once we reach null, we **backtrack** to the parent node and then go right. We keep doing this until we have visited every node in the tree. This is the essence of depth-first search.

As the name implies, we go as deep as possible before we backtrack (normally using recursion, through an iterative approach is possible). The space complexity would be O(h), where h is the height of the tree (log(n) if balanced, n if skewed).

There are three ways to traverse a tree using depth-first search:

- Inorder
- Preorder
- Postorder



Source: Neetcode

Trees | Depth-First Search (DFS)

Depth First Search (DFS) is one of the most common algorithms in coding interviews. It is commonly used to traverse trees and graphs

If we want to traverse an entire tree, we have to visit every node (O(n) time). One way to accomplish this is by using depth-first search

The idea is we pick a direction and keep following pointers as far down left as we can go until reaching null.

Then, we **backtrack** to the parent node and then go right until we have visited every node in the tree

⇒ Go as deep as possible before we backtrack

⇒ The space complexity would be O(h), where h is the height of the tree (log(n) if balanced, n if skewed)

```
def inorder(root):
    if not root:
        return
    inorder(root.left)
    print(root.val)
    inorder(root.right)
```

```
def preorder(root):
    if not root:
        return
    print(root.val)
    preorder(root.left)
    preorder(root.right)
```

```
def postorer(root):
    if not root:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.val)
```

Trees | Breadth-First Search (BFS)

In depth-first search, we prioritized depth. For breath-first search (BFS), we prioritize breadth, visiting all the nodes on one level before moving on to the next level

BFS is also known as level-order traversal when referring to trees, since we visit the nodes level by level

Generally, breadth-first search is implemented iteratively. This requires a specific data structure

A queue data structure, more specifically, a **deque**, allows us to remove elements both from the head and the tail in O(1) time. For BFS we will append elements to the tail and remove elements from the head as we go through each level of the tree from left to right.

```
def bfs(root):
    queue = deque()

    if root:
        queue.append(root)

    level = 0
    while len(queue) > 0:
        print("level: ", level)
        for _ in range(len(queue)):
            curr = queue.popleft()
            print(curr.val)
            if curr.left:
                queue.append(curr.left)
            if curr.right:
                queue.append(curr.right)
        level += 1
```

Source: Neetcode

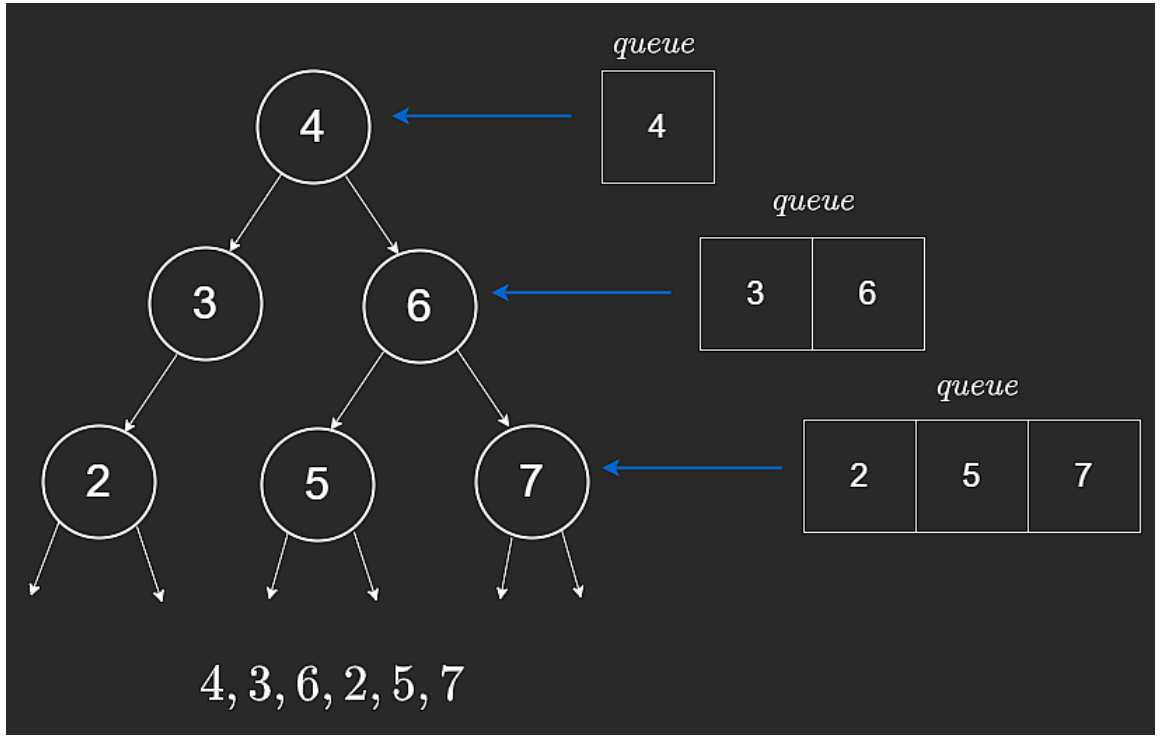
Trees | Breadth-First Search (BFS)

Time

The time complexity of BFS is O(n), where n is the number of nodes in the tree. This is because we visit every node exactly once.

Space

The space complexity of BFS is O(n) where n is the number of nodes in the tree. This is because we will store an entire level of the tree in the queue at a time. In the worst case the last level may be roughly half the size of the tree, so the space complexity is O(n).



Source: Neetcode

Practice Questions 4

http://bit.ly/4cYquOI

Heaps

Heaps | Introduction

A heap is a data structure that offers a **priority queue**. This priority is often the *minimum* or *maximum*.

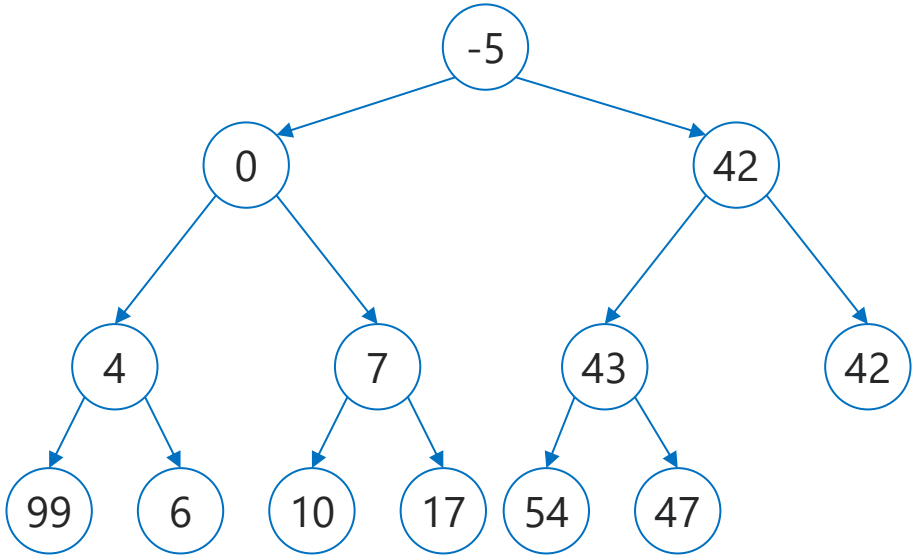
A heap is great at tracking min/max values, but not useful for searching specific elements.

While a heap tends to be implemented in array form, it has tree properties and two invariants:

Structure property: A binary tree is considered *completed* if all nodes have two children, except possibly the last level

Order property: The highest node always satisfies the constraint of the heap, e.g., if it is a min tree, the root node is always the smallest. This is true for every following subtree. Duplicates can exist

.



Heaps | Implementation

Heaps are typically implemented using arrays rather than explicit tree nodes.

It is easiest to index the array starting at 1 instead of 0.

For a node at index i:
leftChild = 2*i (i = current index)
rightChild = 2*i +1
parent = i//2

-5	0	42	4	7	43	42
1	2	3	4	5	6	7

Operation	Big-O time	Notes
Access	O(1)	Only root element
Push	O(log n)	Insert + percolate
Pop	O(log n)	Delete + percolate
Heapify	O(n)	Create a heap from an array

Heaps | Push and Pop

Push (Insert)

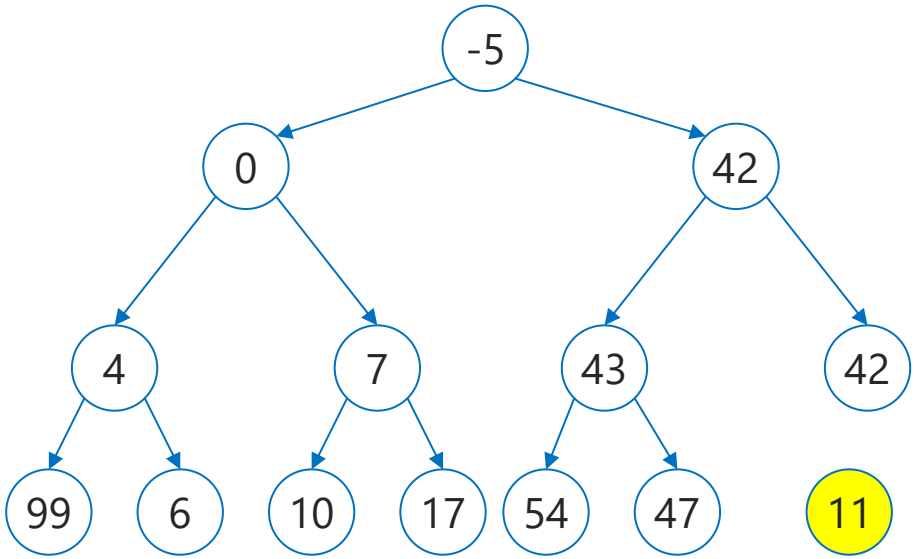
- 1. Add the new element to the end of the array (last position in the tree)
- 2. Compare it with its parent
- 3. If the heap property is violated, swap with parent
- 4. Repeat until the heap property is restored ("percolate up")

Pop (Remove Root)

- 1. Remove the root element (min or max)
- 2. Move the last element to the root
- 3. Compare with children, swap with the smaller (min heap) or larger (max heap) child
- 4. Repeat until heap property is restored ("percolate down")

Both operations are O(log n) because the height of a complete binary tree is log(n). We only traverse one path from root to leaf (or leaf to root).

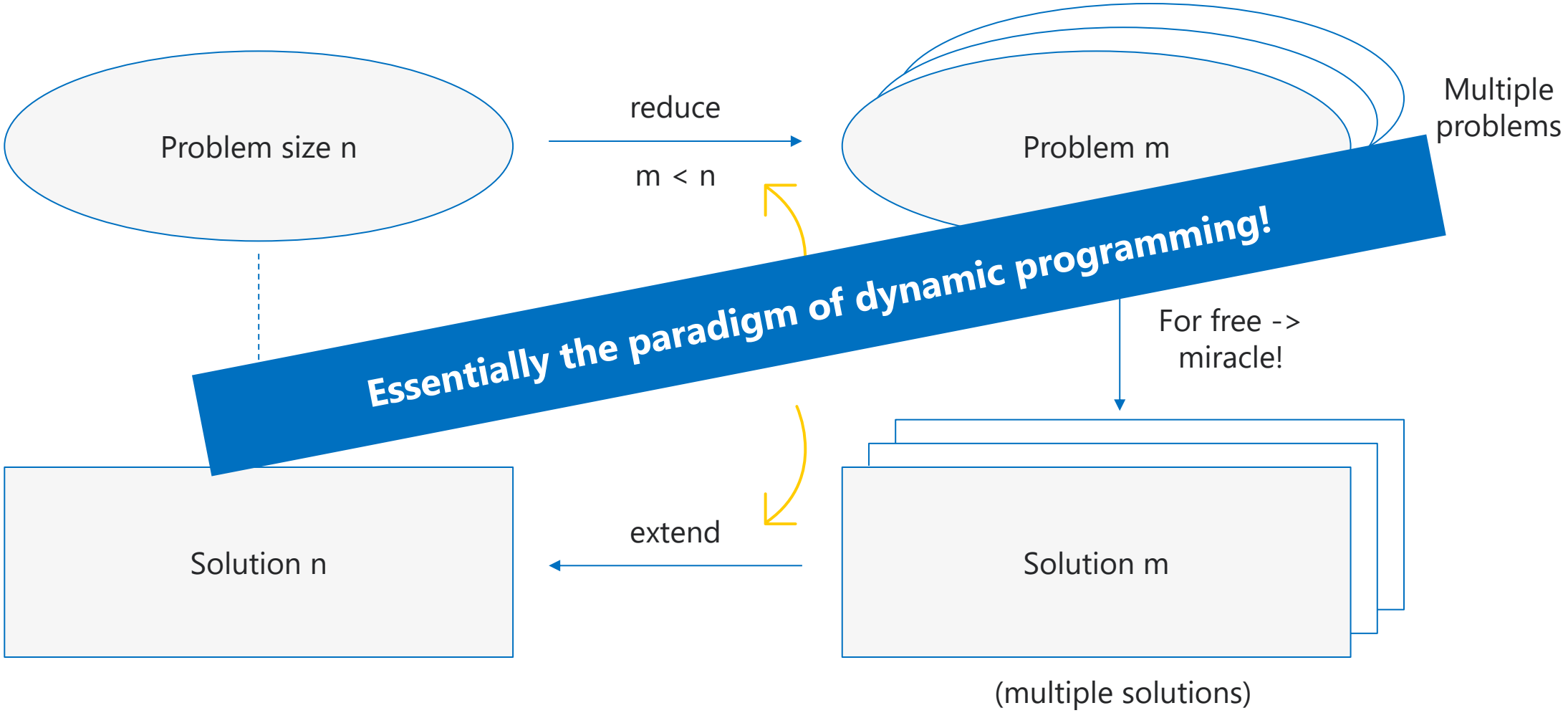
Heaps | Python implementation + insertion example




```python
import heapq

# Min heap by default in Python
# Since Python 3.14, we also
# have max heaps, but most tutorials
# will still lead you to use min heaps
heap: list[int]  = [-5, 43, 0, 7,
                    4, 99, 6, 17, 10,
                    64, 47, 42, 42, 54]

heapq.heapify(heap)

# Push
heapq.heappush(heap, 11)

# Pop (returns min)
min_val = heapq.heappop(heap)

# Access
min_val = heap[0]
```

Backtracking

Backtracking | Using an inductive approach



Source: University of Oxford, Ralf Hinze, Algorithms

Backtracking | Using an inductive approach

Step 1: Base case

Empty list! Permutations = []

Step 2: Extend

Assume all other parts are completed!

Add the new element to all available positions in the list

```
def permute(nums):

   def helper(counter):
      if counter == len(nums):
         res.append(nums.copy())
         return

      for i in range(counter, len(nums):
         nums[counter], nums[i] = nums[i], nums[counter]
         helper(start+1)
         nums[counter], nums[i] = nums[i], nums[counter]

    res = []
    helper(0)
    return res
```

Source: Neetcode

Practice Questions 5

http://bit.ly/4cYquOI

Graphs

Graphs | Introduction

A graph is a data structure made up of nodes (like list nodes and tree nodes), where the nodes may be connected by pointers.

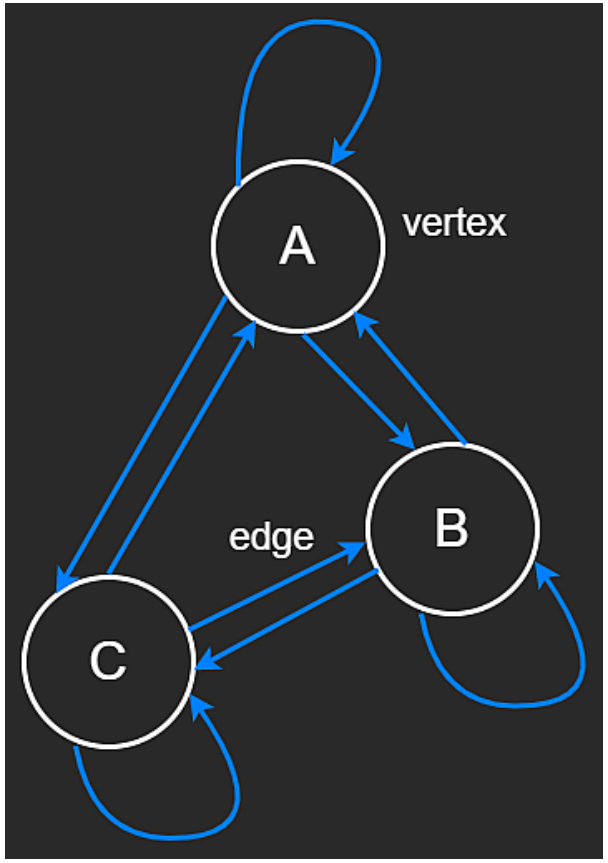
In graphs, nodes are referred to as **vertices** and the pointers connecting these nodes are referred to as **edges**.

There are no restrictions in graphs when it comes to where the nodes are placed, and how the edges connect those nodes together. A node could have a variable number of edges.

Graphs can be directed or undirected

Most commonly, graphs are represented using the following:

- Matrix
- Adjacency Matrix
- Adjacency List

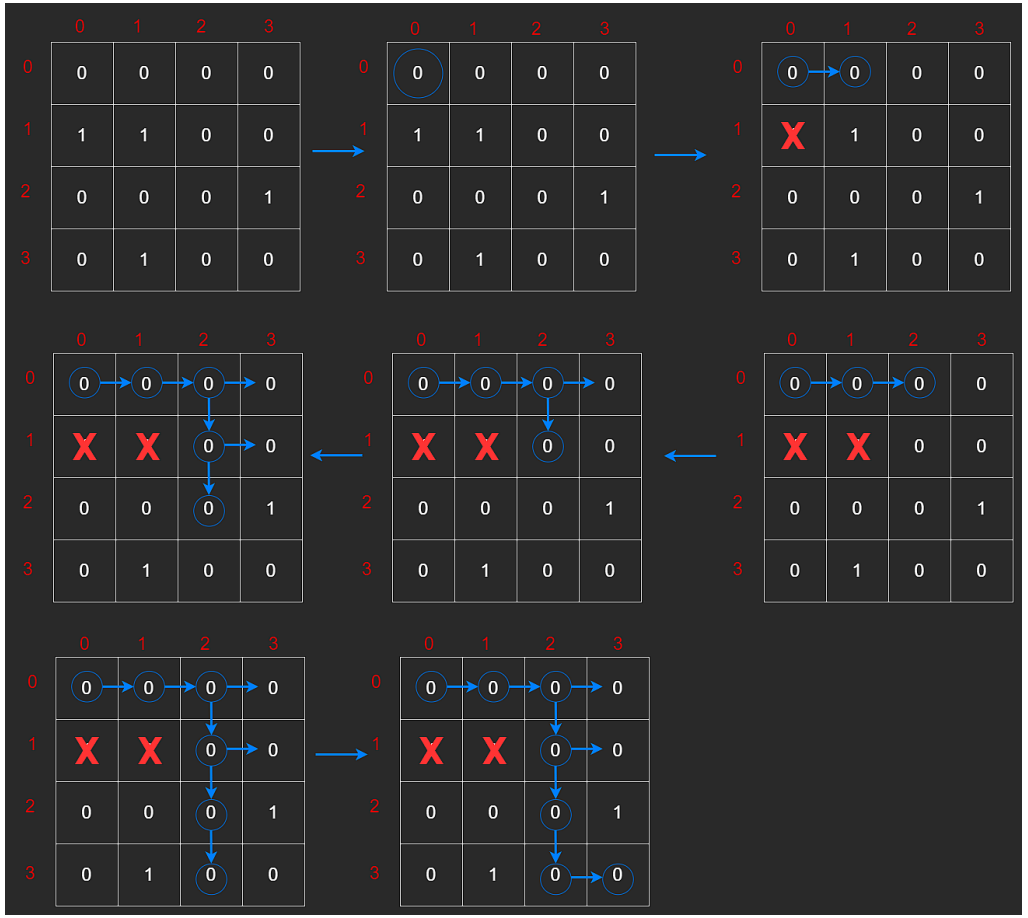


Source: Neetcode

Graphs | Matrix DFS

```python
# Matrix (2D Grid)
grid = [
[0, 0, 0, 0],
[1, 1, 0, 0],
[0, 0, 0, 1],
[0, 1, 0, 0]
] # Count paths (backtracking)

def dfs(grid, r, c, visit):
    ROWS, COLS = len(grid), len(grid[0])
    if (min(r, c) < 0 or
        r == ROWS or
        c == COLS or (
        r, c) in visit or
        grid[r][c] == 1):
        return 0
    if r == ROWS - 1 and c == COLS - 1:
         return 1

    visit.add((r, c))

    count = 0
    count += dfs(grid, r + 1, c, visit)
    count += dfs(grid, r - 1, c, visit)
    count += dfs(grid, r, c + 1, visit)
    count += dfs(grid, r, c - 1, visit)

    visit.remove((r, c))
    return count
```

Find non-blocked (0) route to finish

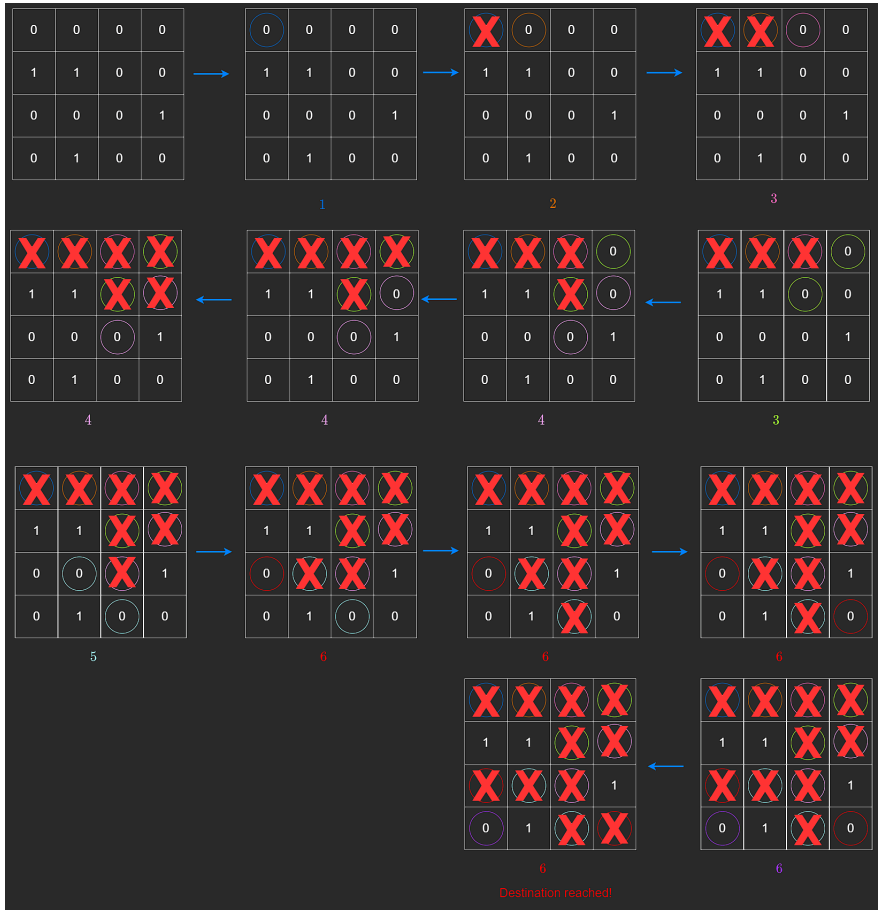


Source: Neetcode

Graphs | Matrix BFS

```python
# Shortest path from top left to bottom right
def bfs(grid):
    ROWS, COLS = len(grid), len(grid[0])
    visit = set()
    queue = deque()
    queue.append((0, 0))
    visit.add((0, 0))

    length = 0
    while queue:
        for i in range(len(queue)):
            r, c = queue.popleft()
            if r == ROWS - 1 and c == COLS - 1:
                return length

            neighbors = [[0, 1], [0, -1], [1, 0], [-1, 0]]
            for dr, dc in neighbors:
                if (min(r + dr, c + dc) < 0 or
                    r + dr == ROWS or
                    c + dc == COLS or
                    r + dr, c + dc) in visit or
                    grid[r + dr][c + dc] == 1):
                    continue
                queue.append((r + dr, c + dc))
                visit.add((r + dr, c + dc))
        length += 1
```

Find non-blocked (0) route to finish



Source: Neetcode

Practice Questions 6

http://bit.ly/4cYquOI

Dynamic programming

Dynamic Programming | Introduction

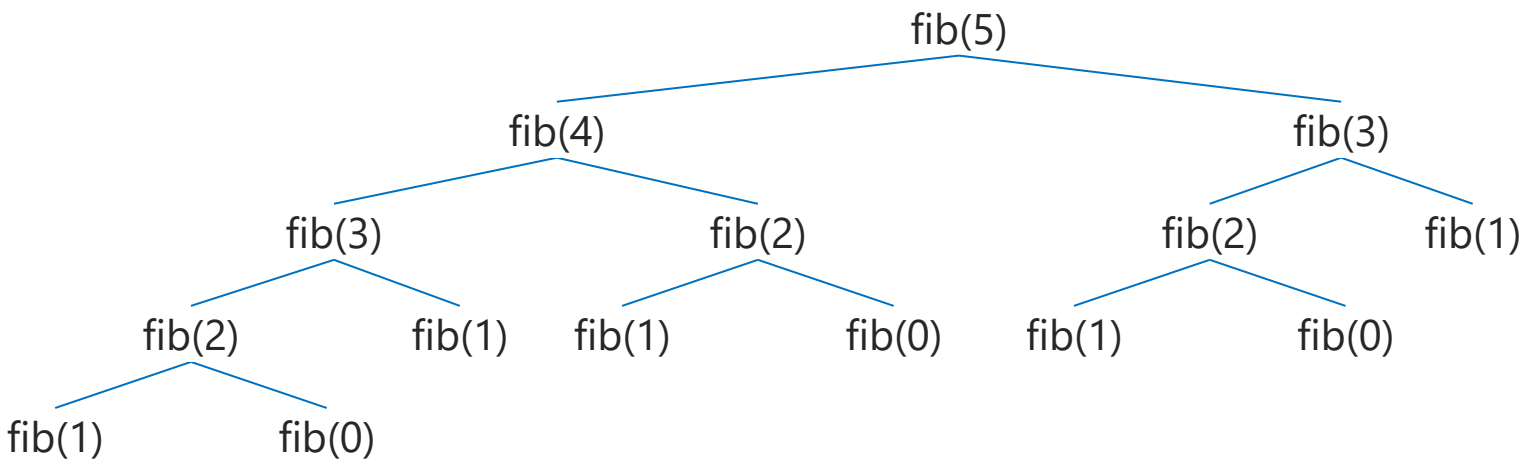
Backtracking is a great approach to solving recursive problems – but a naïve recursive implementation can be exponentially slow!

Dynamic Programming (DP) is a method that solves complex problems by breaking them down into simpler (overlapping) subproblems *and* storing their solutions to avoid redundant computation

Naïve (recursive) approach

```
def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Example for n = 5:



Source: Neetcode

Dynamic Programming | DFS with caching

Dynamic Programming applies look-up tables to pre-computed function results

Trading time for space: Decreasing running time while increasing space consumption

Naïve (recursive) approach

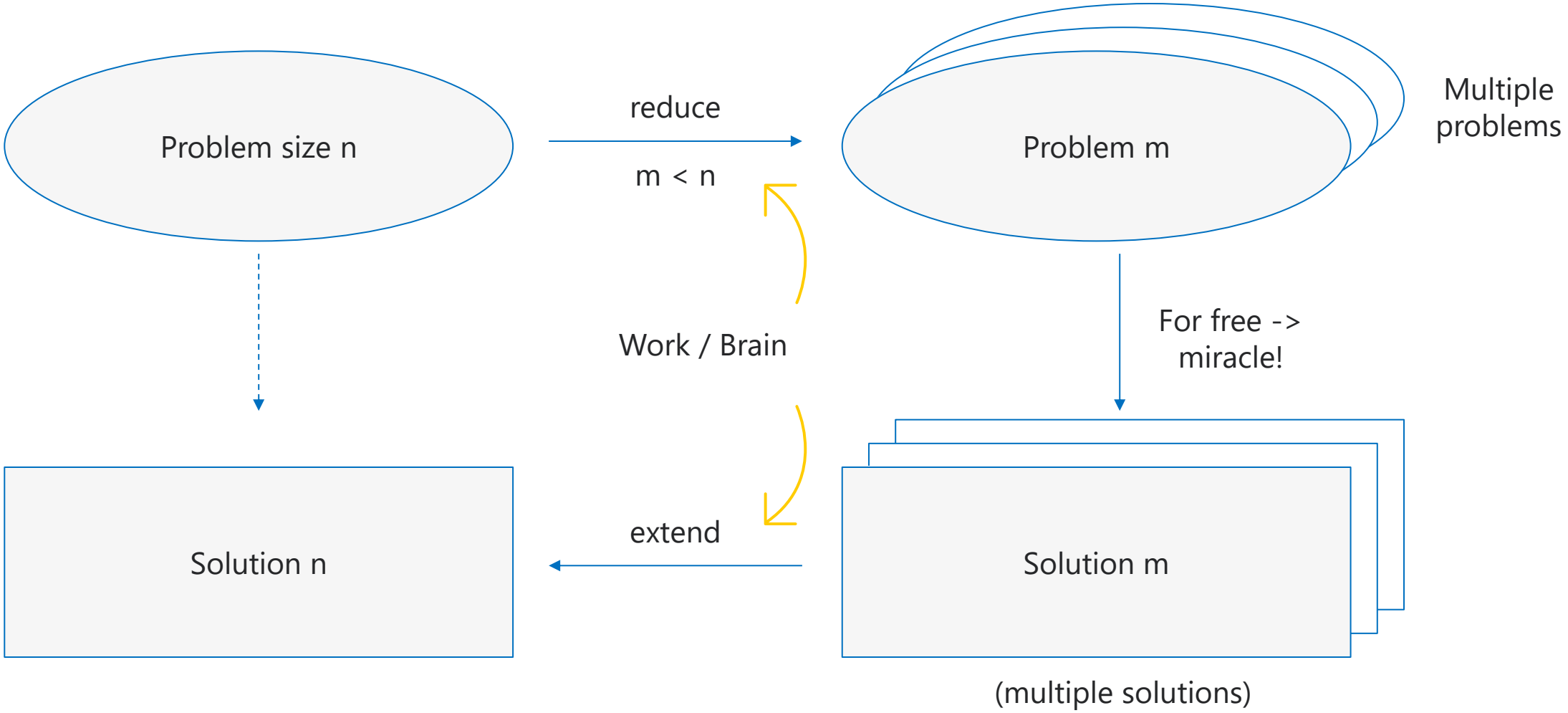
```
def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

DP approach

```
def fibonacci(n: int) -> int:
    cache = [-1] * n

    def dfs(i: int):
        if i >= n:
            return i == n
        if cache[i] != -1:
            return cache[i]
        cache[i] = dfs(i + 1) + dfs(i + 2)
        return cache[i]

    return dfs(0)
```

Source: Neetcode

Dynamic Programming | Reminder of using an inductive approach



Source: University of Oxford, Ralf Hinze, Algorithms

Dynamic Programming | DFS with caching – Example2

Coin change problem: Given coins of different denominations, what is the minimum number of coins to provide change to any given amount?

Naïve "solution": Greedy approach.

Use the largest denomination first, until we must switch to smaller ones.

However, this does not always work:

Coins: 1, 3, 4

Change given: 6

Greedy: 4 + 1 + 1 = 6 -> Three coins

However, 3 + 3 = 6 is the correct solution.

Recursive naïve solution (working):

DP solution (efficient and working):

Naïve (recursive) approach

```
def coinChange(n: int) -> int:
    if n == 0:
       return 0
    if n < 0:
       return -1
    return 1 + min(
        coinChange(n-1),
        coinChange(n-3),
        coinChange(n-4))
```

DP solution

```
def coinChange(coins: List[int], amount: int) -> int:
    dp = [amount + 1] * (amount + 1)
    dp[0] = 0

     for a in range(1, amount + 1):
         for c in coins:
             if a - c >= 0:
                 dp[a] = min(dp[a], 1 + dp[a - c])
     return dp[amount] if dp[amount] != amount + 1 else -1
```

Source: Neetcode

Practice Questions 7

http://bit.ly/4cYquOI